

High-Level Synthesis for Multi-Core Systems

Behavioral code

C/C++
SystemC

HLS compiler

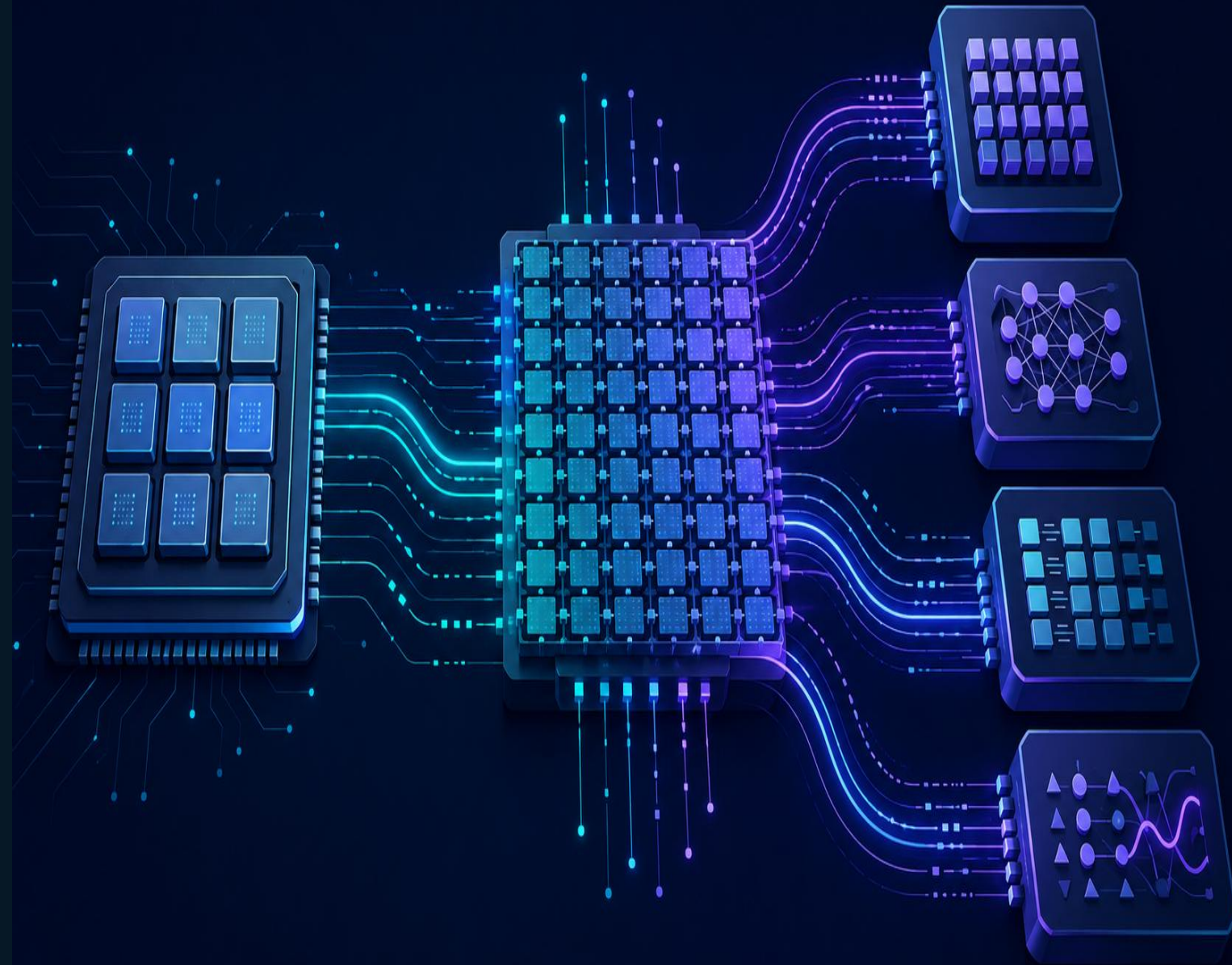
Schedule
Bind

RTL hardware

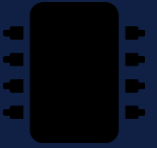
VHDL
Verilog

Md Mostafizur Rahman

Electronic Engineering • Hochschule Hamm-Lippstadt



Introduction



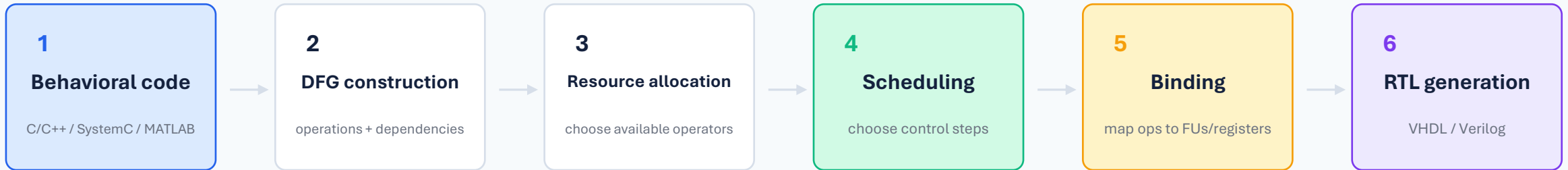
What is HLS?

- ✓ HLS automates RTL generation from high-level descriptions (C/C++, SystemC, MATLAB)
- ✓ Analogous to software compilation — but for hardware
- ✓ Tools: AMD Xilinx Vitis HLS, ScaleHLS, BAMBU
- ✓ Multi-core HLS adds:
 - PE partitioning
 - Communication scheduling
 - Inter-PE resource sharing

6-Stage HLS Flow

- ① Behavioral Compilation
- ② Data-Width Analysis
- ③ Resource Allocation
- ④ Operation Scheduling
- ⑤ Resource Binding
- ⑥ RTL Code Generation

From Code to RTL



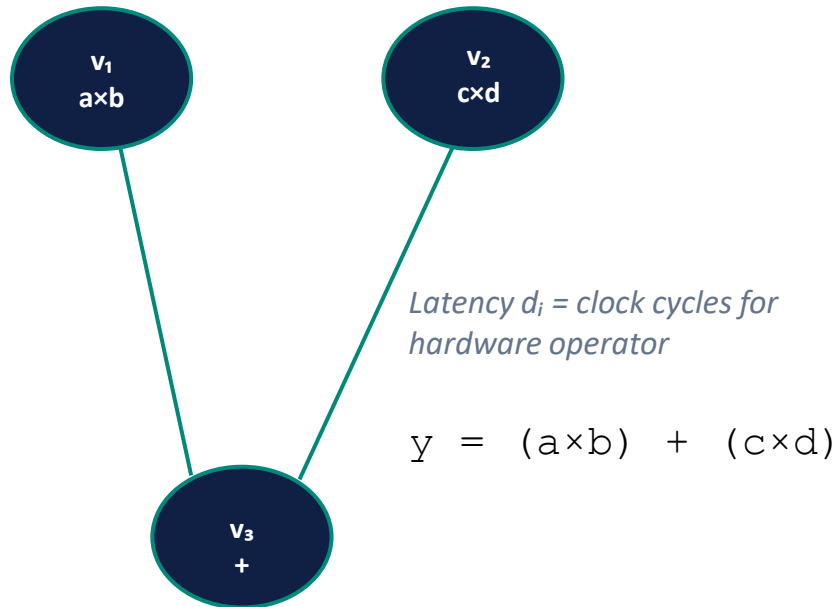
Multi-core extension



Mathematical Foundations

Data-Flow Graph (DFG)

$G = (V, E)$ — nodes = operations, edges = data dependencies



ASAP · ALAP · Mobility

ASAP

Earliest possible control step for each operation

ALAP

Latest possible step under latency constraint λ

Mobility

$t^{\text{ALAP}}(v_i) - t^{\text{ASAP}}(v_i) \rightarrow$ scheduling flexibility

Parallel Speedup

$$S(k) = T_1 / T_k$$

T_1 = single-core latency

T_k = k -core latency

Ideal: $S(k) \leq k$ (communication overhead reduces this)

Multi-Core HLS Formal Model

π



Partition

Assign each DFG node $v_i \rightarrow \text{PE } P_j$

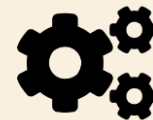
s



Schedule

Assign each node a control step $\in \mathbb{Z}^+$

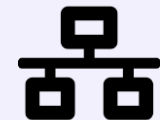
β



Bind

Map each operation $\rightarrow$ functional unit FU

δ



Communicate

Schedule inter-PE data transfers

Minimise total latency Λ , chip area A , power P while satisfying communication, resource, and timing constraints.

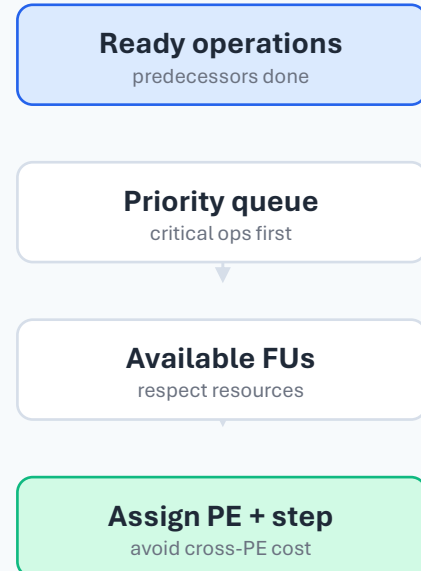
Communication Constraint: $s(v_j) \geq s(v_i) + d_i + \delta_{ij}$

Communication Constraint: $\text{start}(\text{consumer}) \geq \text{finish}(\text{producer}) + \text{communication delay}$

Algorithm Focus: Scheduling + Binding

Scheduling and binding should not be treated as two isolated problems.

List-based scheduler



Priority function

$$\text{Priority}(v_i) = 1/\text{mobility}(v_i) + \alpha_1 \cdot \text{width}(v_i) + \alpha_2 \cdot |\text{succ}(v_i)| + \text{PE-affinity}$$

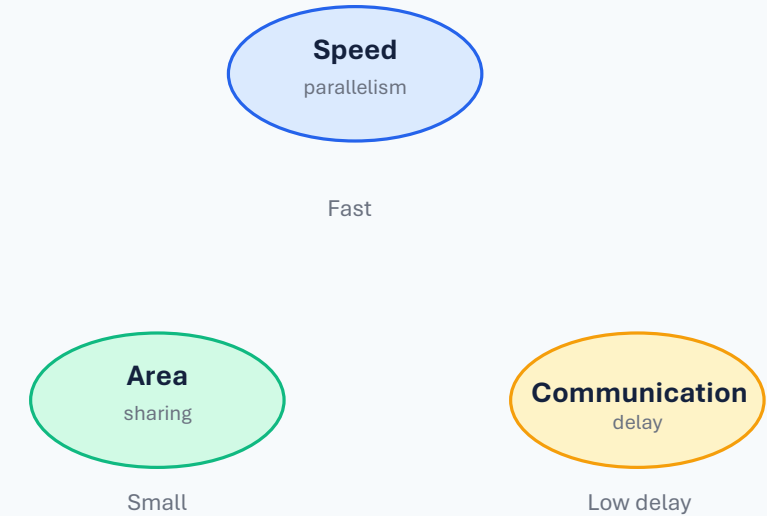
Low mobility = critical path. PE-affinity reduces unnecessary communication.

Binding cost

$$\chi(y_i, n_j) = \omega_{op} + \omega_{path} + \text{communication term}$$

Choose not only the cheapest operator, but also the cheapest data path.

Design trade-off



Experiment Strategy

1

C++ scheduling model

Shows operation → step → PE assignment

`sched.cpp`

2

POSIX multi-core run

Measures practical speedup and overhead

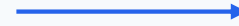
`matrix_pthread.c`

3

Vivado FPGA synthesis

Compares separate vs. shared datapath area

VHDL + utilization report



Together they prove

DFG scheduling maps work to PEs

Real speedup is useful but not ideal

Resource sharing can reduce FPGA LUTs

Best design balances all three

C++ List-Based Scheduler

A small 2×2 matrix multiplication DFG is scheduled onto two processing elements.

```
// 2x2 matrix multiply DFG scheduling
struct Operation {
    int id, latency, step=-1, core=-1;
    vector<int> preds;
};

bool predecessorsDone(op, step) {
    for (int p : op.preds)
        if (ops[p].step+ops[p].latency > step)
            return false;
    return true; }

// Greedy loop: assign ready ops to PE
for (int step=1; !allScheduled(ops); step++)
    for (auto& op : ops)
        if (predecessorsDone(op, step))
            assign to first PE with free FU;
```

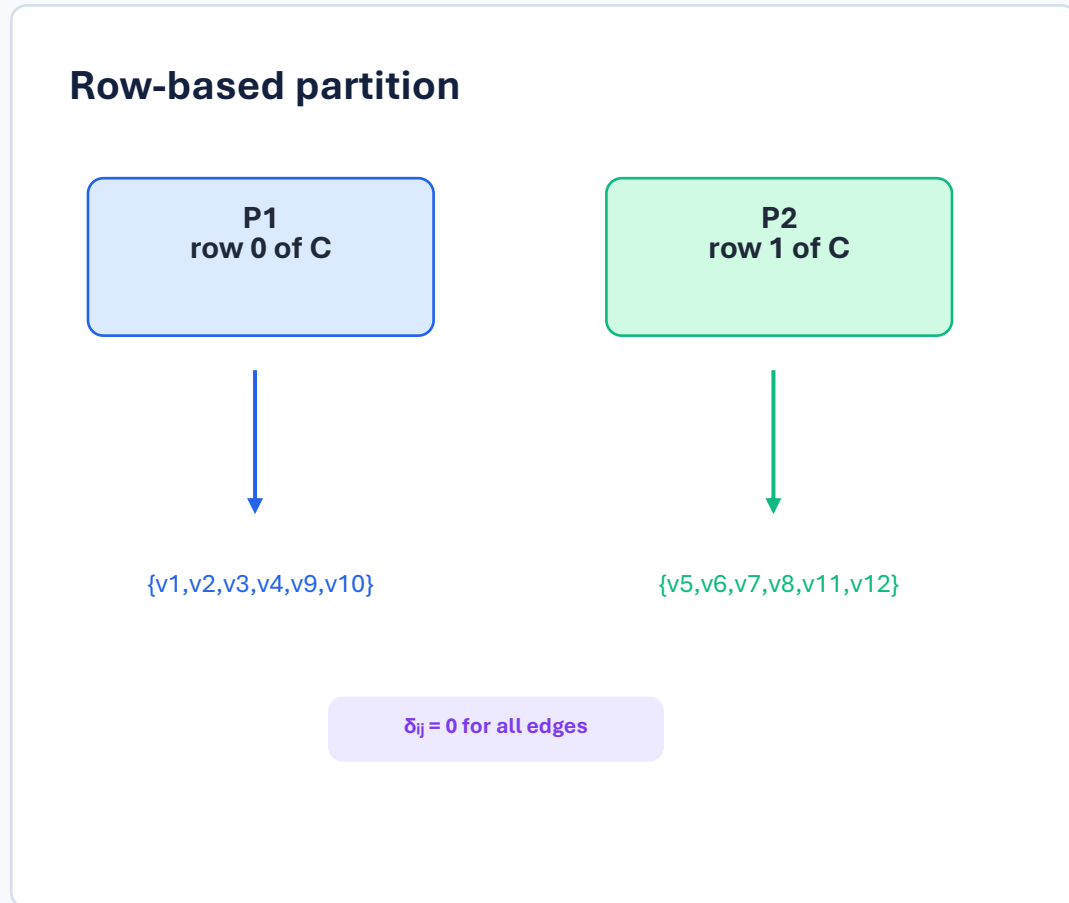
Observed output

Step	PE1	PE2
1	v1, v2	v3, v4
2	v5, v6	v7, v8
3	v9, v10	—
4	v11, v12	—

- ❑ DFG: 2×2 matrix multiply (12 ops)
- ❑ 2 Processing Elements
- ❑ 2 Functional Units per PE
- ❑ Multiplication: latency = 2 cycles
- ❑ Addition: latency = 1 cycle

2×2 Example: Communication-Aware Partitioning

For the 2×2 matrix product, row-based partitioning removes inter-PE data transfers.



Two-core schedule

Step	PE1 (Row 0)	PE2 (Row 1)
1 – 2	$v1 \times v2$ (mult)	$v5 \times v6$ (mult)
3	$v9$ (add)	$v11$ (add)
4 – 5	$v3 \times v4$ (mult)	$v7 \times v8$ (mult)
6	$v10$ (add)	$v12$ (add)

Ideal speedup: $S(2) = T_1/T_2 = 12/6 = 2.0$

POSIX Multi-Core Simulation

```
// matrix_pthread.c (N=64, CLK_US=8)
void *pe_worker(void *arg) {
    // Each thread = one PE
    int rows = N/K;
    // Row-partitioned matrix multiply
    for (i=start; i<end; i++)
        for (j=0; j<N; j++)
            C[i][j] += A[i][k]*B[k][j];
    // Simulate HW latency
    usleep(rows*(MUL+ADD)*CLK_US);
}

// Measure wall-clock for K=1,2,4
double T1 = run_test(1);
double T2 = run_test(2);
double T4 = run_test(4);
S(K) = T1 / TK;
```

Hardware Model Mapping

pthread

Processing Element (PE)

row block

DFG partition

usleep()

hardware operator latency

pthread_join

synchronization δ_{ij}

AMD Ryzen 7

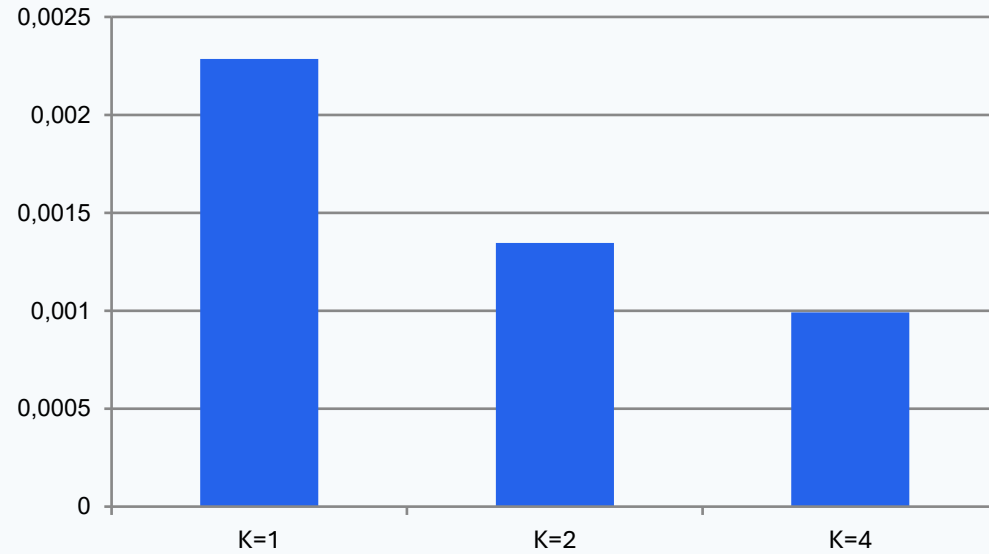
8-core target machine

Test Configuration

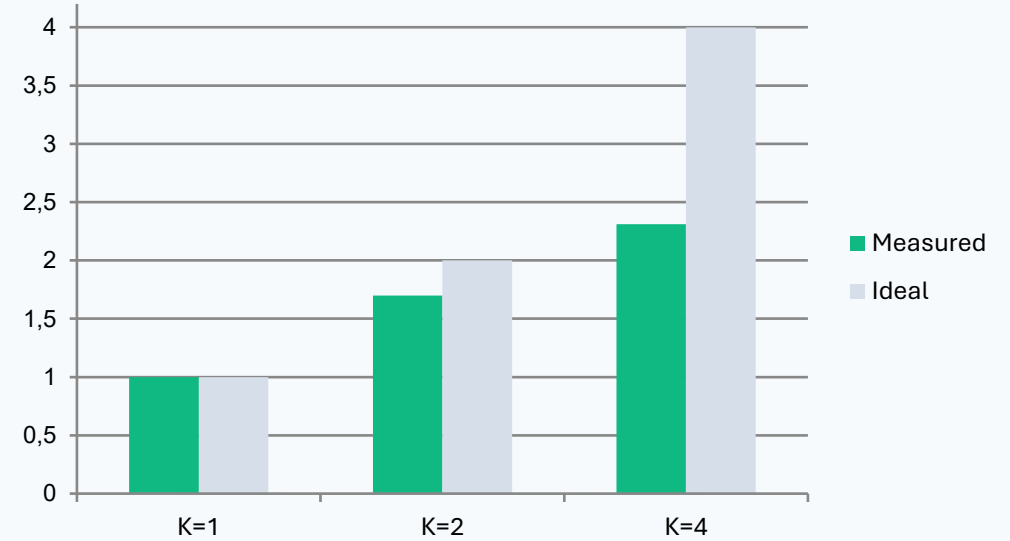
- ✓ N = 64 × 64 matrix
- ✓ 6 runs — median reported
- ✓ Linux CLOCK_MONOTONIC timer
- ✓ gcc -O2 -pthread

POSIX Results

Linux pthreads model processing elements and expose real overhead.



Execution time ↓



Measured vs. ideal speedup

1.00×

K=1 (baseline)

$T_1 = 0.002286 \text{ s}$

1.70×

K=2 (two PEs)

$T_2 = 0.001346 \text{ s}$

2.31×

K=4 (four PEs)

$T_4 = 0.000992 \text{ s}$

FPGA Resource-Sharing Experiment

Same interface, different internal datapath: duplicated hardware vs. shared hardware.

Separate datapath

Mode 0: $y = (a \times b) + (c \times d) \rightarrow 2 \text{ multipliers} + 1 \text{ adder}$



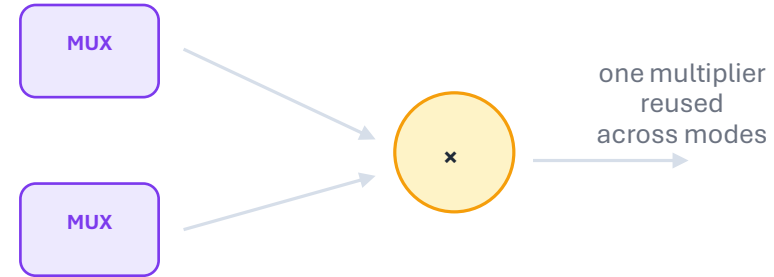
Mode 1: $y = (a + b) \times (c + d) \rightarrow 2 \text{ adders} + 1 \text{ multiplier}$



mode selects output

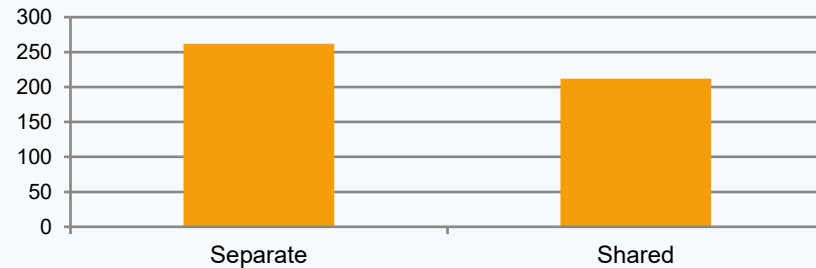
Higher area, more independent logic

Shared-resource design



```
shared_mult <= op1 * op2;
```

Lower area, resource sharing



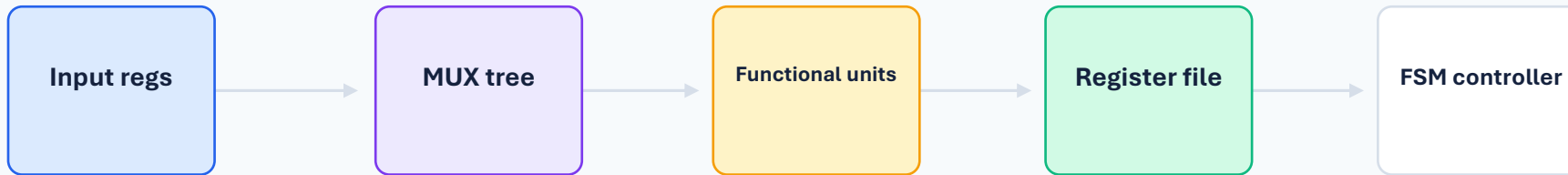
19.1% LUT reduction

262 $\rightarrow$ 212 LUTs, same 50 IO pins

- ❑ All 6 functional units always present in the netlist
- ❑ No sharing — operations use dedicated hardware

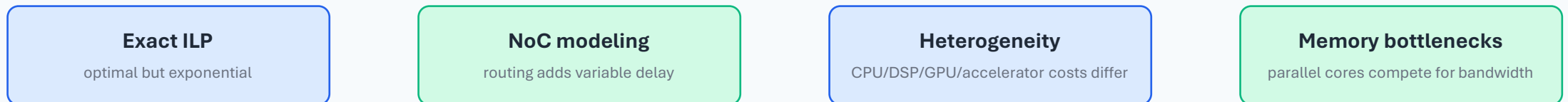
RTL Output and Practical Limits

Per-PE RTL structure

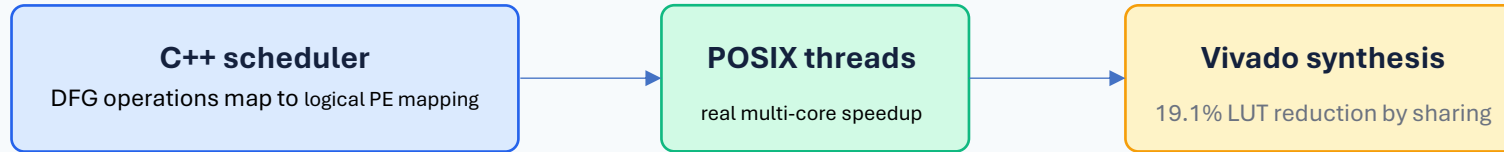


Operators + registers + MUXes + FSM control signals become VHDL/Verilog hardware.

Limitations / open challenges



Conclusion



- ✓ DFG scheduling maps operations to cycles and PEs.
- ✓ Pthread experiment shows real speedup, but not ideal scaling.
- ✓ Vivado experiment shows 19.1% LUT reduction from sharing.
- ✓ Best HLS designs balance parallelism, communication, and area.

Final Decision: Parallelism improves speed, but communication and resource sharing decide whether the hardware is efficient.

References and Questions

- [1] J. P. Elliott, Understanding Behavioral Synthesis, Kluwer, 2000.
- [2] P. Coussy and A. Morawiec, High-Level Synthesis: From Algorithm to Digital Circuit, Springer, 2008.
- [3] S. Gupta et al., SPARK: A Parallelizing Approach to HLS of Digital Circuits, Springer, 2004.
- [4] G. De Micheli, Synthesis and Optimization of Digital Circuits, McGraw-Hill, 1994.
- [5] E. Casseau and B. Le Gal, Multi-mode application-specific cores based on HLS, VLSI Journal, 2012.
- [6] P. G. Paulin and J. P. Knight, Force-Directed Scheduling, IEEE TCAD, 1989.
- [7] A. M. Sllame and V. Drabek, List-based scheduling for HLS, DSD, 2002.
- [8] M. Stephenson et al., Bitwidth analysis, PLDI, 2000.
- [9] R. Nane et al., Survey and evaluation of FPGA HLS tools, IEEE TCAD, 2016.
- [10] H. Ye et al., ScaleHLS, HPCA, 2022.

**Thank you
Questions?**